

# 深入解析 printf 和 scanf 的格式符

## (含完整详解和避坑点)

### 一、printf 格式符完整详解（核心格式符+修饰符，标★高频）

printf 的核心是\*\*“格式化输出”，格式符的作用是指定“如何解析数据并展示”，支持基础格式符\*\*+修饰符组合，覆盖所有输出场景。下面按类别分核心格式符（标★必记），再讲常用修饰符（扩展功能），附示例，新手能直接用。

#### （一）printf 核心基础格式符（按用途分类，★高频必记）

所有格式符均为 C 标准定义，编译器全兼容，覆盖 99%日常开发，冷门的用到再查即可。

| 数据类型 | 格式符  | 详细作用（printf）                         | 高频示例（输出结果）                                   |
|------|------|--------------------------------------|----------------------------------------------|
| 整型★  | %d   | 有符号十进制整数<br>(int/short, short 可加%hd) | printf("%d", 97); →97                        |
|      | %u   | 无符号十进制整数<br>(unsigned int)           | printf("%u", -1); →4294967295<br>(32 位)      |
|      | %ld  | 有符号长整型<br>(long)                     | printf("%ld", 100000L); →100000              |
|      | %lld | 有符号超长整型<br>(long long)               | printf("%lld", 1e18LL); →1000000000000000000 |
|      | %o   | 无符号八进制整数                             | printf("%o", 8); →10                         |

|         |       |                                     |                                     |
|---------|-------|-------------------------------------|-------------------------------------|
|         |       | (无前导 0)                             |                                     |
|         | %x/%X | 无符号十六进制整数 (小写/大写, 无 0x)             | printf("%x",97)<br>;→61             |
| 字符/字符串★ | %c    | 单个字符 (char, 解析 ASCII 码为字符) ★        | printf("%c",97)<br>;→a              |
|         | %s    | 字符串 (char*, 指向首地址, 遇\0 结束) ★        | printf("%s","Java");→Java           |
| 浮点型★    | %f    | 十进制浮点 (float/double 通用, 默认 6 位小数) ★ | printf("%f",3.14);→3.140000         |
|         | %lf   | 双精度浮点 (和%f 等价, printf 中可混用)         | printf("%lf",3.14);→3.140000        |
|         | %.2f  | 保留 2 位小数 (修饰符+%f, 高频组合) ★           | printf("%.2f",3.1415);→3.14         |
|         | %e/%E | 科学计数法浮点 (小写/大写 e)                   | printf("%e",123.45);→1.234500e+02   |
|         | %g/%G | 简化浮点 (自动去多余 0, 结合%f/%e)             | printf("%g",3.14000);→3.14          |
| 地址/指针★  | %p    | 打印内存地址/指针 (带 0x, 十六进制, 标准写法) ★      | printf("%p",&a)<br>;→0x7ffeefbff5ac |

|        |    |                                       |                                                     |
|--------|----|---------------------------------------|-----------------------------------------------------|
| 特殊格式符★ | %% | 输出%符号本身<br>(转义, 无参数)<br>★             | <code>printf("%d<br/>= %d", 97);</code> →%d<br>= 97 |
|        | %n | 冷门: 获取已输出<br>字符数 (存到 int*<br>变量, 无输出) | 几乎不用, 了解即可                                          |

## (二) printf 常用格式修饰符 (加在格式符前, 扩展功能★)

修饰符不能单独使用, 必须和基础格式符搭配, 用来控制输出宽度、精度、对齐方式、补 0 等, 是格式化输出的关键, 新手掌握以下 5 个就够了。

| 修饰符  | 作用说明                                                           | 搭配示例                   | 输出结果 (重点看格式)                                                                      |
|------|----------------------------------------------------------------|------------------------|-----------------------------------------------------------------------------------|
| 数字 n | 固定输出宽度 n, 默认右对齐, 不足补空格★                                        | <code>%5d</code>       | <code>printf("%5d", 97);</code> → 97 (2 空格+97)                                    |
| 0n   | 固定宽度 n, 不足补 0 (替代空格), 仅对数值有效★                                  | <code>%05d</code>      | <code>printf("%05d", 97);</code> →00097                                           |
| .n   | 1.浮点型: 保留 n 位小数★<br>2.字符串: 截取前 n 个字符                           | <code>%.2f/%.3s</code> | <code>printf("%.2f", 3.1415);</code> →3.14<br><code>printf("%.3s", "Python");→Pyt |
| -n   | 固定宽度 n, 左对齐 (默认右对齐)★                                           | <code>%-5d</code>      | <code>printf("%-5d", 97);</code> →97<br>(97+2 空格)                                 |
| l/L  | 修饰长整型/浮点型<br><code>%ld=long, %lld=long long<br>%Lf=long double | <code>%ld</code>       | <code>printf("%ld", 100L);</code> →100                                            |

### (三) printf 高频组合示例（日常开发最常用，直接抄）

```
C
#include <stdio.h>
int main() {
    int a = 97;
    float f = 3.1415926;
    char str[] = "Python";
    // 高频组合：固定宽度+补 0
    printf("补 0: %06d\n", a); // 000097
    // 高频组合：保留 2 位小数（浮点型必用）
    printf("保留 2 位小数: %.2f\n", f); // 3.14
    // 高频组合：左对齐+固定宽度
    printf("左对齐: %-8s 结束\n", str); // Python 结束（Python+2 空格）
    // 高频组合：右对齐+固定宽度
    printf("右对齐: %8s 结束\n", str); //   Python 结束（2 空格+Python）
    return 0;
}
```

## 二、scanf 格式符完整详解（核心格式符+修饰符，标★高频）

scanf 的核心是\*\*\*“格式化输入”，格式符的作用是指定“如何解析输入数据并存储到变量”，和 printf 的格式符大部分通用，但有 3 个核心差异（新手必踩坑），且 scanf 的修饰符功能更简单（主要是限制长度、跳过空白\*\*）。

### 关键前提：scanf 传参规则★

scanf 的参数必须是「变量的地址」（因为要把输入值写入变量内存），这是和 printf 的本质区别，牢记 2 条规则：

1. 普通变量（int/char/float）：加&取地址，如 `scanf("%d", &a);`;
2. 字符串数组（char str[N]）/字符指针（char\*）：不用加&（数组名/指针本身就是地址），如 `scanf("%s", str);`。

### （一）scanf 核心基础格式符（按用途分类，★高频必记）

和 printf 通用的格式符不再重复说明，重点标注和 printf 的差异（△），标★的是高频

必记。

| 数据类型    | 格式符  | 详细作用<br>(scanf)                 | 高频示例（正<br>确写法）          | 与 printf 的差<br>异△                                |
|---------|------|---------------------------------|-------------------------|--------------------------------------------------|
| 整型★     | %d   | 读取有符号十<br>进制整数到<br>int/short    | scanf("%d",<br>&a);     | 无差异                                              |
|         | %u   | 读取无符号十<br>进制整数到<br>unsigned int | scanf("%u",<br>&u);     | 无差异                                              |
|         | %ld  | 读取长整型到<br>long                  | scanf("%ld"<br>, &l);   | 无差异                                              |
|         | %lld | 读取超长整型<br>到 long long           | scanf("%lld"<br>, &ll); | 无差异                                              |
| 字符/字符串★ | %c   | 读取单个字符<br>到 char★               | scanf(" %c"<br>, &ch);  | △会读取空格/<br>换行，需加空<br>格跳过                         |
|         | %s   | 读取字符串到<br>char[]，遇空<br>白结束★     | scanf("%s",<br>str);    | △自动在末尾<br>加\0，不读空<br>白                           |
| 浮点型★    | %f   | 读取浮点型到<br>float★                | scanf("%f",<br>&f);     | △printf 中%f<br>兼容 double，<br>scanf 仅适配<br>float  |
|         | %lf  | 读取浮点型到<br>double★               | scanf("%lf"<br>, &d);   | △scanf 必<br>用%lf 读<br>double，printf<br>可混用%f/%lf |
|         | %Lf  | 读取浮点型到<br>long double           | scanf("%Lf"<br>, &ld);  | 无差异                                              |

|       |    |                        |                     |          |
|-------|----|------------------------|---------------------|----------|
| 地址/指针 | %p | 读取地址字符串到 void*         | scanf("%p", &addr); | 极少用，了解即可 |
| 特殊    | %% | 无此用法<br>(scanf 不需要输出%) | -                   | 无此格式符    |

## （二）scanf 常用格式修饰符（仅 3 个，高频★）

scanf 的修饰符远少于 printf，核心只有 3 个，主要解决\*\*\*“限制字符串长度（防缓冲区溢出）”和“跳过空白”\*\*的问题，是新手必学的避坑技巧。

| 修饰符  | 作用说明（scanf 专属/高频）                 | 搭配示例             | 作用效果★                                                                 |
|------|-----------------------------------|------------------|-----------------------------------------------------------------------|
| 数字 n | 限制字符串读取的<br>最大长度 n-1（留<br>1 位存0）★ | %5s              | scanf("%5s", str); 输入<br>Python→仅存<br>Pytho（5 个字符<br>+\\0），防缓冲区溢<br>出 |
| *    | 跳过指定类型的输<br>入数据（不存储到<br>变量）       | 跳过 10 个数字<br>%*d | scanf("%*d %d", &a); 输入 10<br>20→a=20（跳过<br>10）                       |
| 空格   | 跳过所有空白字符<br>（空格/制表/换<br>行）★       | %c               | scanf(" %c", &ch); 输入\n<br>a→ch='a'（跳过换<br>行）                         |

## （三）scanf 新手必避坑示例（3 个高频错误，直接抄正确写法）

```
C
#include <stdio.h>
int main() {
```

```
int a;
char ch;
char str[10];
float f;
double d; // 重点: double 类型
// 坑 1: double 用%f 读取 (错误) → 正确: %lf
scanf("%lf", &d); // 必须用%lf, 否则数据乱码
// 坑 2: %c 读取到换行/空格 (错误) → 正确: %c 前加空格
scanf(" %c", &ch); // 跳过所有空白, 只读有效字符
// 坑 3: %s 无长度限制 (缓冲区溢出, 错误) → 正确: 加数字限制长度
scanf("%9s", str); // 数组长度 10, 限制读 9 个字符, 留 1 位存\0
// 正确组合示例
scanf("%d %d %9s %f %lf", &a, str, &f, &d);
return 0;
}
```

### 三、printf & scanf 格式符核心差异对照表（新手必背，避坑★）

这是最关键的部分，把两者的**核心差异**整理成表格，记熟这 5 点，能避开 90% 的格式符混用坑。

| 对比维度  | printf（格式化输出）                                                          | scanf（格式化输入）                                                 |
|-------|------------------------------------------------------------------------|--------------------------------------------------------------|
| 传参规则★ | 传变量值/表达式（如 <code>printf("%d", a+1)</code> ）                            | 传变量地址（普通变量加 <code>&amp;</code> ，字符串数组不加 <code>&amp;</code> ） |
| 浮点型★  | <code>%f</code> 兼容 float/double， <code>%lf</code> 和 <code>%f</code> 等价 | <code>%f</code> 仅适配 float，读 <b>double 必须用 %lf</b> （核心坑）      |
| 字符%c★ | 直接输出字符，无空白问题                                                           | 会读取空格/换行/制表，需加空格跳过空白                                         |
| 字符串%s | 输出到\0 结束，无长度限制问题                                                       | 读取到空白结束， <b>必须加数字限制长度</b> （防缓冲区溢出）                           |
| 修饰符用途 | 丰富（控制宽度、精度、对齐等）                                                        | 简单（仅限制长度、跳过空白）                                               |

|      |             |          |
|------|-------------|----------|
|      | 对齐、补 0)     | 数据、跳过空白) |
| %%符号 | 用于输出%本身（转义） | 无此用法，不需要 |

## 四、总结（核心要点，5 分钟背会，覆盖所有开发）

### 1. %p 和%d 打印地址的核心结论

- 地址是指针类型，不是 int 类型，%d 能打印只是“强转解读”，会截断、出负数、跨平台不兼容；
- 打印地址/指针，唯一标准、安全的写法是%p，带 0x 前缀+十六进制+适配所有平台，禁止用%d/%x 替代。

### 2. printf 核心必记（5 个格式符+4 个修饰符）

- 格式符：%d（整型）、%c（字符）、%s（字符串）、%.2f（浮点保留 2 位）、%p（地址）、%%（输出%）；
- 修饰符：%nd（固定宽度）、%0nd（补 0）、[%.nf](%.nf)（精度）、%-nd（左对齐）。

### 3. scanf 核心必记（5 个格式符+3 个修饰符+2 条传参规则）

- 格式符：%d（整型）、%c（字符，加空格）、%s（字符串，限长度）、%f（float）、%lf（double）；
- 修饰符：%ns（限字符串长度）、%\*d（跳过数据）、%c（跳过空白）；
- 传参规则：普通变量加&，字符串数组不加&。

### 4. 两者最核心的 3 个差异（避坑）

1. scanf 读 double 必须用%lf，printf 中%f/%lf 等价；
2. scanf 的%c 必须加空格跳过空白，printf 不用；
3. scanf 的%s 必须加数字限制长度，printf 不用。

实际开发中，无需死记所有格式符，记住以上核心内容，剩下的冷门场景用到再查 C 标准手册即可——这也是所有程序员的通用做法。

|（注：文档部分内容可能由 AI 生成）